Weekend Interview Prep — Senior SWE, AI & Cloud (GCP)

Oct 29, 2026 · @Abdalla Misbah

What this role really is

They want the first in-house engineer who can take vendor-built apps away from the vendors, run them on GCP, and add AI features. Three themes win this interview: **ownership** (I can inherit messy code and make it mine), **shipping on GCP** (Cloud Run + Cloud Build + CI/CD, end to end), and **practical AI** (RAG and agents with Gemini, built safely for enterprise).

Decoding the context. Dhahran + "trading, risk management, Front/Middle/Back Office" + "commodity trading, energy markets" points strongly to an energy trading business (Aramco Trading Company is headquartered there; confirm with the recruiter). Expect questions about trading-domain apps, data sensitivity, and audit.

The hidden message in the JD. "Foundational hire" + "transition from vendor-led" + "mentoring future developers" means they are hiring a future tech lead. Show you can set standards, not only write code.

Current-events edge. The JD says "Vertex AI", but at Google Cloud Next in April 2026 Google rebranded Vertex AI as the **Gemini Enterprise Agent Platform** (Agent Studio, Agent Runtime, Agent Registry, Agent Identity, Agent Gateway). Existing Vertex services keep working under the new name. Mentioning this casually shows you are current.

Weekend outcome (definition of done by Sunday night):

- [ ] Can explain every term in the Glossary in 2 sentences + one real example
- [ ] A deployed demo: FastAPI + React RAG/agent app on Cloud Run, built by Cloud Build from a Git push
- [ ] A one-page vendor-takeover plan you can describe from memory
- [ ] 6 STAR stories written and rehearsed out loud
- [ ] A 2-minute "tell me about yourself" mapped to the three themes

Weekend schedule

About 20 focused hours: Friday night sets up, Saturday is build-heavy, Sunday is AI + interview rehearsal. Every study block ends by saying the topic out loud as if answering an interviewer.

When	Block	Do	Output
Fri 19:00–20:00	Setup	Create a GCP project (free trial credit), install gcloud, Docker, Node, Python 3.12, Terraform. Enable Cloud Run, Cloud Build, Artifact Registry, Secret Manager, Vertex AI APIs	Working `gcloud` login
Fri 20:00–22:00	Glossary pass 1	Read the Glossary below. Mark each term green (can explain), amber (fuzzy), red (new)	Your red/amber list
Fri 22:00–22:30	Stories	Draft 6 STAR stories (list in the Questions section) as bullet points	Story notes
Sat 09:00–11:00	Build 1: backend	FastAPI service, Pydantic models, `/health`, `/ask`, pytest tests, Dockerfile. Run in Docker locally	Green tests, container runs
Sat 11:00–12:30	Build 2: Cloud Run	Push image to Artifact Registry, deploy to Cloud Run, set env vars + Secret Manager, service account with least privilege	Public HTTPS URL
Sat 13:30–15:30	Build 3: CI/CD	`cloudbuild.yaml` (test → build → push → deploy), trigger on Git push to `main`, dev vs prod services	Push = auto deploy
Sat 15:30–17:30	Build 4: frontend	React + TypeScript (Vite) chat page calling the API; serve from a second Cloud Run service or from FastAPI static	Working UI

When	Block	Do	Output
Sat 17:30–19:00	Study: architecture + security	REST design, SQL vs NoSQL, auth (OAuth2/OIDC, IAP), OWASP top 10, logging/monitoring on GCP	Red terms turned amber
Sun 09:00–11:30	Build 5: RAG	Upload docs to Cloud Storage, chunk + embed with `gemini-embedding-001`, store vectors (pgvector or in-memory for demo), retrieve → Gemini answer with citations	`/ask` answers from your docs
Sun 11:30–13:00	Build 6: agent	Add 2 tools (e.g. `get_price`, `calc_exposure`) via Gemini function calling or Google ADK	Agent calls a tool
Sun 14:00–15:00	Stretch: Terraform	Define the Cloud Run service + service account + bucket in Terraform; `terraform plan`	IaC file to show
Sun 15:00–16:30	Vendor takeover	Learn the playbook section; pick any open-source FastAPI repo and do a 30-min mock audit	1-page audit notes
Sun 16:30–18:30	Mock interview	Answer every question in the Questions section out loud; record yourself; redo weak ones	Rehearsed answers
Sun 18:30–19:30	Domain	Front/Middle/Back office, ETRM, PnL, VaR, exposure, hedging (Glossary)	Can describe a trade's lifecycle
Sun 20:00	Stop	Final checklist, sleep	Calm

If time is short, the minimum viable weekend is: Build 1–3 + Build 5 + mock interview. That alone proves the three themes.

Glossary: every term in the JD

Each row = what it is in plain words + a line you can say in the interview. Learn the "Say this" column by heart.

Business and domain

Term	What it is	Say this
Front Office	The people who make money: traders, originators, sales. Apps: trade capture, pricing, market data screens	"Front office apps must be fast and available during market hours; latency and uptime matter most."
Middle Office	Risk and control: risk management, PnL reporting, limit monitoring, compliance	"Middle office needs correct, auditable numbers: exposure, VaR, PnL explained."
Back Office	Operations after the trade: confirmations, settlement, invoicing, accounting	"Back office is about reconciliation and integration with ERP like SAP."
Commodity trading	Buying/selling physical crude, refined products, LNG, chemicals, plus derivatives (futures, swaps) to hedge	"Physical trades have logistics: cargoes, vessels, nominations; paper trades are hedges."
ETRM / CTRM	Energy/Commodity Trading and Risk Management systems (e.g. Allegro, Endur/Openlink, RightAngle)	"Often the system of record; new apps integrate around it rather than replace it."
Trade lifecycle	Deal capture → validation → confirmation → risk/PnL → scheduling/logistics → settlement → accounting	Be able to draw this on a whiteboard
PnL	Profit and loss, realized (closed) and unrealized (mark-to-market)	"Daily PnL must reconcile with the ETRM."
Mark-to-market (MtM)	Valuing open positions at today's market price	
Exposure / position	How much of a commodity you are long or short, by product, tenor, location	
VaR	Value at Risk: worst expected loss at a confidence level (e.g. 95%, 1 day)	"Historical-simulation VaR is a classic batch job; good fit for Cloud Run Jobs."

Term	What it is	Say this
Hedging	Taking an opposite derivative position to reduce price risk	
Market data	Prices from Platts, Argus, ICE, CME; licensed and expensive	"Market data licences restrict who sees what; enforce access in the API."

Software engineering

Term	What it is	Say this
Enterprise application	Internal business software with many users, SSO, audit, integrations	
API	A contract other programs call. REST = resources over HTTP with verbs GET/POST/PUT/PATCH/DELETE and status codes	"I design APIs contract-first with OpenAPI, versioned (/v1), idempotent writes, pagination."
Backend services	Server code holding business logic and data access	
End-to-end (full-stack)	Backend + frontend + database + integrations, all yours	
Python	Main language. Know: typing, async/await, generators, context managers, dataclasses, packaging (uv/poetry)	
FastAPI	Async Python API framework on Starlette + Pydantic. Auto OpenAPI docs, dependency injection, validation	"FastAPI for APIs and AI services; Django when I need admin, ORM and auth out of the box."
Django	Batteries-included Python web framework: ORM, admin, auth, migrations	

Term	What it is	Say this
Pydantic	Data validation via type hints; also great for validating LLM structured outputs	
React	UI library built from components and state; hooks (useState, useEffect)	
TypeScript	JavaScript with static types; catches bugs at compile time	
Software architecture	How parts fit: layers, modules, services, data flow, trade-offs	"Start as a modular monolith; split into services only where scaling or ownership demands it."
Monolith vs microservices	One deployable vs many small independently deployable services	
SQL database	Relational, tables, joins, ACID transactions (PostgreSQL, Cloud SQL, AlloyDB, BigQuery for analytics)	"Trades and money go in SQL: transactions and constraints."
NoSQL	Document/key-value/wide-column (Firestore, Bigtable, MongoDB). Flexible schema, horizontal scale	"Firestore for app state, chat sessions, configs."
Application integration	Connecting systems: REST, files (SFTP), message queues, ETL, webhooks	
Security	AuthN (who are you), AuthZ (what can you do), encryption, secrets, input validation, least privilege, OWASP Top 10	"Secrets in Secret Manager, never in code; service accounts with minimum roles; SSO via OIDC."
Code review	Reading others' code for correctness, design, security, readability	
Coding standards	Agreed style + rules: linters (ruff, ESLint), formatters (black, Prettier), naming, structure	

Term	What it is	Say this
Reusable components / accelerators	Templates, shared libraries, starter repos so new apps start fast	"A golden-path template: FastAPI + React + Dockerfile + cloudbuild.yaml + Terraform."
Technical documentation	README, architecture decision records (ADRs), API docs, runbooks	

Google Cloud and DevOps

Term	What it is	Say this
GCP	Google Cloud Platform	
Cloud Run	Serverless containers: you give a container, Google runs and scales it (to zero), HTTPS URL included. Services (HTTP) and Jobs (batch)	"Cloud Run is my default: no cluster to manage, pay per use, revisions allow instant rollback and traffic splitting."
Cloud Build	Google's CI/CD runner. Steps in `cloudbuild.yaml`, each a container; triggered by Git push	
Cloud Storage (GCS)	Object storage in buckets: files, documents, model inputs	
Artifact Registry	Stores Docker images and packages	
Secret Manager	Stores API keys, passwords; mounted into Cloud Run as env vars or files	
IAM / service account	Identity and permissions; each service runs as its own service account with least privilege	
IAP	Identity-Aware Proxy: puts Google SSO in front of an internal app with no code	"Quick way to make internal tools company-only."

Term	What it is	Say this
Cloud Logging / Monitoring / Trace	Logs, metrics, alerts, dashboards, distributed tracing (Google Cloud Observability)	
Pub/Sub	Managed messaging for event-driven systems	
Cloud SQL / AlloyDB	Managed PostgreSQL/MySQL; AlloyDB is high-performance Postgres with built-in vector search	
BigQuery	Serverless data warehouse for analytics	
GKE	Google Kubernetes Engine	
Git	Version control: branches, pull requests, merge vs rebase, tags	"Trunk-based with short-lived branches, PR reviews, protected main."
Docker	Packages an app + dependencies into an image; runs as a container	
CI/CD	Continuous Integration (every push is built and tested) + Continuous Delivery/Deployment (automatically released)	
Automated testing	Unit, integration, end-to-end, contract tests; pytest, Jest/Vitest, Playwright	
Release process	Versioning, change approval, deploy, verify, rollback	"Canary on Cloud Run: 10% traffic to the new revision, watch errors, then 100%."
Environments	dev → test/UAT → prod, ideally separate GCP projects	
Monitoring & logging	Structured JSON logs, metrics, alerts, SLOs	

Term	What it is	Say this
Performance optimization	Profiling, caching, DB indexes, connection pooling, concurrency settings, cold-start tuning (min instances)	
Terraform	Infrastructure as Code tool: describe cloud resources in HCL, `plan` then `apply`, state file	
Infrastructure as Code (IaC)	Infra defined in versioned code, not console clicks; reviewable and repeatable	
Kubernetes	Container orchestration: pods, deployments, services, ingress	"I'd pick GKE over Cloud Run only for long-running stateful workloads or special networking."
Event-driven architecture	Services react to events (trade booked → risk recalculated) via Pub/Sub, Eventarc	
Cloud-native microservices	Small stateless containerized services, auto-scaled, observable	
Data engineering	Pipelines moving/cleaning data: Dataflow, BigQuery, Composer (Airflow)	

AI

Term	What it is	Say this
Vertex AI (now Gemini Enterprise Agent Platform)	Google's managed AI platform: model access, Model Garden (200+ models incl. Claude, Gemma), tuning, evaluation, Agent Runtime	

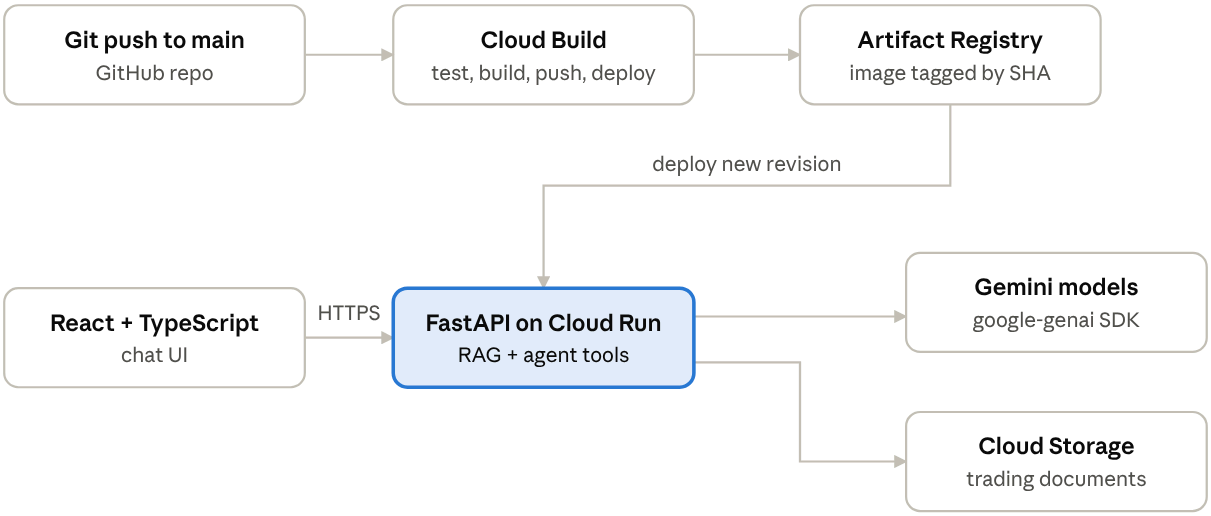
Term	What it is	Say this
Gemini API	Google's LLM family. Current tiers: Pro (hard reasoning), Flash (fast default), Flash-Lite (cheap, high volume). Python SDK is `google-genai`	"Flash for most traffic, Pro where accuracy justifies cost."
LLM	Large Language Model: predicts text; can reason, summarize, extract, write code	
LLM application	A product built around LLM calls: prompts, context, tools, guardrails, evaluation	
Prompt engineering	Writing instructions, examples, and output formats for the model	
Embeddings	Turn text into vectors so similar meaning = nearby vectors	
Vector database	Stores embeddings for similarity search (pgvector, AlloyDB, Vertex Vector Search)	
RAG	Retrieval-Augmented Generation: retrieve relevant chunks from your documents, put them in the prompt, the model answers grounded in them with citations	"RAG fixes stale knowledge and hallucination for company data, without fine-tuning."
Chunking	Splitting documents into pieces (e.g. 500–1000 tokens with overlap) before embedding	
Hybrid search / reranking	Combine keyword (BM25) + vector search, then rerank top results for precision	
AI agent	An LLM in a loop that plans, calls tools (APIs, DB queries), observes results, and continues until done	"Agents need guardrails: limited tools, human approval for actions, logging every step."

Term	What it is	Say this
Function calling / tool use	The model returns a structured call (`get_price(product='Brent')`); your code runs it and returns the result	
ADK	Google's open-source Agent Development Kit (Python, TS, Go, Java); deploys to Agent Runtime, Cloud Run, or GKE	
MCP	Model Context Protocol: a standard way to expose tools/data to any agent	
A2A	Agent2Agent protocol: agents talking to agents	
Intelligent automation	Using AI to automate business workflows, e.g. extract fields from trade confirmations and match them	
Hallucination	Model confidently states false info; mitigate with RAG, citations, validation, evaluation	
Evaluation (evals)	Test sets of questions + expected answers, scored automatically (incl. LLM-as-judge) before each release	"I treat prompts like code: versioned and regression-tested."
Guardrails	Input/output filtering, PII redaction, Model Armor, allow-listed tools	
Data scientist vs AI engineer vs software engineer	DS builds models/analysis; AI engineer builds LLM/ML systems; you productionize, integrate and own the apps	

Hands-on project: "Trading Desk Assistant"

Build one small app that touches almost every required skill: a FastAPI + React app on Cloud Run that answers questions from trading documents (RAG) and calls tools (agent), deployed by Cloud Build on every Git push. In the interview, say "I built this over the weekend to prepare" and show the URL and repo.

A Git push ships the app; Cloud Run serves RAG and agent answers



project architecture · CI/CD path and request path

Top row is the delivery path you build Saturday; bottom row is the request path you build Sunday.

Repo layout

```
trading-assistant/
  api/
    main.py          # FastAPI app
    rag.py           # chunk, embed, retrieve
    agent.py         # tools + function calling
    tests/test_api.py
    requirements.txt
    Dockerfile
  web/               # React + TypeScript (Vite)
  infra/main.tf      # Terraform
  cloudbuild.yaml
  README.md          # how to run, architecture, decisions
```

Step 1 — Setup (Fri, 1 h)

```
gcloud auth login && gcloud auth application-default login
gcloud projects create trading-assistant-<you> && gcloud config set project
trading-assistant-<you>
gcloud services enable run.googleapis.com cloudbuild.googleapis.com \
  artifactregistry.googleapis.com secretmanager.googleapis.com
aiplatform.googleapis.com storage.googleapis.com
gcloud artifacts repositories create apps --repository-format=docker --
location=me-central2
```

`me-central2` is the Dammam region, a good talking point on Saudi data residency. If a service is unavailable there for your account, use `europe-west1`.

Step 2 — FastAPI backend (Sat morning)

```
# api/main.py
import os
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from google import genai

app = FastAPI(title="Trading Desk Assistant", version="1.0")
client = genai.Client(vertexai=True,
project=os.environ["GOOGLE_CLOUD_PROJECT"], location="global")
MODEL = os.getenv("MODEL", "gemini-3.5-flash")  # check exact ID in the model
list

class AskIn(BaseModel):
    question: str

class AskOut(BaseModel):
    answer: str
    sources: list[str] = []

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/v1/ask", response_model=AskOut)
def ask(body: AskIn):
    if not body.question.strip():
```

```
        raise HTTPException(422, "empty question")
    from rag import retrieve
    chunks = retrieve(body.question, k=4)
    context = "\n\n".join(f"[{c['source']}] {c['text']}" for c in chunks)
    prompt = ("Answer only from the context. Cite sources in brackets. "
              "If the answer is not in the context, say you don't know.\n\n"
              f"Context:\n{context}\n\nQuestion: {body.question}")
    resp = client.models.generate_content(model=MODEL, contents=prompt)
    return AskOut(answer=resp.text, sources=sorted({c["source"] for c in
chunks}))
```

```
# api/tests/test_api.py
from fastapi.testclient import TestClient
from main import app

def test_health():
    assert TestClient(app).get("/health").json() == {"status": "ok"}
```

```
# api/Dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
USER 1000
CMD exec uvicorn main:app --host 0.0.0.0 --port ${PORT:-8080}
```

`requirements.txt`: `fastapi`, `uvicorn[standard]`, `pydantic`, `google-genai`, `google-cloud-storage`, `numpy`, `pytest`, `httpx`.

Know why: `USER 1000` (don't run as root), `$PORT` (Cloud Run injects it), slim image (faster cold start).

Step 3 — Deploy to Cloud Run (Sat late morning)

```
gcloud iam service-accounts create rag-api-sa
gcloud projects add-iam-policy-binding $PROJECT --member=serviceAccount:rag-
api-sa@$PROJECT.iam.gserviceaccount.com --role=roles/aiplatform.user
gcloud run deploy rag-api --source api --region me-central2 \
  --service-account rag-api-sa@$PROJECT.iam.gserviceaccount.com \
  --set-env-vars GOOGLE_CLOUD_PROJECT=$PROJECT --allow-unauthenticated
```

Then practise: `gcloud run services describe`, view logs in Cloud Logging, deploy a second revision and split traffic 90/10 with `gcloud run services update-traffic`, roll back.

Step 4 — CI/CD with Cloud Build (Sat afternoon)

```
# cloudbuild.yaml
substitutions:
  _REGION: me-central2
  _IMAGE: me-central2-docker.pkg.dev/${PROJECT_ID}/apps/rag-api
steps:
  - id: test
    name: python:3.12-slim
    dir: api
    entrypoint: bash
    args: ["-c", "pip install -r requirements.txt && pytest -q"]
  - id: build
    name: gcr.io/cloud-builders/docker
    args: ["build", "-t", "${_IMAGE}:${SHORT_SHA}", "api"]
  - id: push
    name: gcr.io/cloud-builders/docker
    args: ["push", "${_IMAGE}:${SHORT_SHA}"]
  - id: deploy
    name: gcr.io/google.com/cloudsdktool/cloud-sdk
    entrypoint: gcloud
    args: ["run", "deploy", "rag-api", "--image", "${_IMAGE}:${SHORT_SHA}",
"--region", "${_REGION}"]
images: ["${_IMAGE}:${SHORT_SHA}"]
```

Connect your GitHub repo in Cloud Build → Triggers → on push to `main`. Then add a second trigger for a `dev` branch deploying `rag-api-dev`. That is your dev/prod environment story. Grant the Cloud Build service account `roles/run.admin` and `roles/iam.serviceAccountUser`.

Step 5 — React frontend (Sat late afternoon)

`npm create vite@latest web -- --template react-ts`. One page: textarea, Ask button, answer, list of sources. Call `fetch(API_URL + "/v1/ask", {method: "POST", ...})`. Add CORS middleware in FastAPI for the web origin. Deploy as a second Cloud Run service (nginx serving `dist/`) or build it into FastAPI static files.

Step 6 — RAG (Sun morning)

Put 3–5 PDFs/text files in a bucket: e.g. a public commodity glossary, an Incoterms summary, a made-up "trading desk policy".

```
# api/rag.py  (in-memory vector store: fine for a demo; say pgvector/AlloyDB
for prod)
import numpy as np
from main import client

INDEX: list[dict] = []  # {text, source, vec}

def chunk(text: str, size=800, overlap=150):
    for i in range(0, len(text), size - overlap):
        yield text[i:i + size]

def embed(texts: list[str]) -> np.ndarray:
    r = client.models.embed_content(model="gemini-embedding-001",
contents=texts)
    return np.array([e.values for e in r.embeddings])

def add_document(text: str, source: str):
    parts = list(chunk(text))
    for t, v in zip(parts, embed(parts)):
        INDEX.append({"text": t, "source": source, "vec": v /
np.linalg.norm(v)})

def retrieve(q: str, k=4):
    qv = embed([q])[0]; qv /= np.linalg.norm(qv)
    scored = sorted(INDEX, key=lambda c: -float(c["vec"] @ qv))
    return scored[:k]
```

Load the bucket's files at startup (`google-cloud-storage`). Be ready to explain: chunk size trade-off, why cosine similarity, why the "say you don't know" instruction, how you'd evaluate (10 Q/A pairs, check citation correctness), and the managed alternative (Vertex AI RAG Engine / Vector Search).

Step 7 — Agent with tools (Sun late morning)

```
# api/agent.py
from google.genai import types
from main import client, MODEL

PRICES = {"brent": 82.4, "wti": 78.1, "dubai": 80.9}  # fake data for demo

def get_price(benchmark: str) -> dict:
    """Return the latest price in USD/bbl for a crude benchmark (brent, wti,
dubai)."""
    return {"benchmark": benchmark, "usd_per_bbl":
PRICES.get(benchmark.lower())}

def calc_exposure(volume_bbl: float, price: float) -> dict:
    """Return USD exposure for a volume in barrels at a price."""
    return {"usd": volume_bbl * price}

def run_agent(question: str) -> str:
    resp = client.models.generate_content(
        model=MODEL, contents=question,
        config=types.GenerateContentConfig(tools=[get_price, calc_exposure]),
    )  # the SDK runs the functions automatically and loops
    return resp.text
```

Try: "What is our USD exposure on a 500,000 barrel Brent cargo?" The model should call `get_price` then `calc_exposure`. Expose it as `POST /v1/agent`. Stretch: rebuild it with Google ADK and `adk web` for the dev UI.

Step 8 — Terraform (stretch, Sun afternoon)

```
# infra/main.tf
provider "google" { project = var.project  region = "me-central2" }

resource "google_service_account" "api" { account_id = "rag-api-sa" }

resource "google_storage_bucket" "docs" {
  name = "${var.project}-docs"  location = "ME-CENTRAL2"
uniform_bucket_level_access = true
}

resource "google_cloud_run_v2_service" "api" {
  name     = "rag-api"
  location = "me-central2"
  template {
    service_account = google_service_account.api.email
    containers { image = var.image }
    scaling { min_instance_count = 0  max_instance_count = 3 }
  }
}
```

(Put each attribute on its own line in the real file.) Run `terraform init && terraform plan`. You don't need to apply; being able to explain plan/apply/state and remote state in a GCS bucket is enough.

What to say about the project (60 seconds)

"Containerized FastAPI service on Cloud Run with its own least-privilege service account, Gemini via the google-genai SDK, RAG over documents in Cloud Storage with citations, two function-calling tools for an agent flow, Cloud Build pipeline that tests, builds, pushes to Artifact Registry and deploys on every push, separate dev and prod services, Terraform for the infra. For production I'd add IAP/SSO, pgvector or Vector Search, evals in the pipeline, and Model Armor for guardrails."

Vendor takeover playbook

This is the part most candidates under-prepare, and it is the core of the job. Your message: "I'd make each vendor app buildable, deployable and understood by us before changing anything, then raise quality step by step."

Assessment checklist (run on every inherited app)

Area	Questions to answer	Quick tools
Access & ownership	Do we own the repo, cloud project, domains, licences, credentials? Any code only on the vendor's laptop?	Repo + IAM audit
Build & run	Can a new engineer run it locally in under 1 hour from the README? Can we deploy without the vendor?	Fresh clone test
Architecture	Components, data flow, integrations, single points of failure	Draw the diagram; ADR for gaps
Code quality	Structure, duplication, complexity, dead code, consistent style	ruff, ESLint, SonarQube, radon
Tests	Coverage, do they run in CI, are they meaningful	pytest --cov
Security	Hard-coded secrets, outdated dependencies, injection, auth checks, over-privileged service accounts	gitleaks, pip-audit, npm audit, Bandit, Artifact Analysis scanning
Data	Schema, migrations, backups, PII/commercially sensitive data, retention	
Operations	Logs, alerts, dashboards, runbooks, on-call, SLAs	Cloud Monitoring
AI-specific	Prompts versioned? Evals exist? Model versions pinned? Cost per request? Guardrails?	
Documentation	README, API docs, architecture, known issues	

Area	Questions to answer	Quick tools
Commercial	Contract IP clauses, handover obligations, support period, warranty	Work with procurement

Knowledge transfer method

1. Shadow: vendor demos, walks through code, deploys while you watch.
2. Reverse shadow: you deploy and fix a small bug while the vendor watches.
3. Solo with safety net: you own releases; vendor on call for a fixed period.
4. Exit: access revoked, documentation and runbooks signed off.

Record sessions, keep a "questions log", and make handover acceptance criteria part of the contract (builds from clean clone, tests green in our CI, runbook exists).

30-60-90 day plan (have this ready to say)

- **First 30 days:** meet stakeholders and vendors, inventory every app (owner, users, criticality, tech stack, vendor), get every repo into company Git, run the checklist on the top 2 critical apps, get one app deploying through our own Cloud Build pipeline.
- **Days 31–60:** write engineering standards (branching, PR review, linting, testing minimums, Dockerfile and `cloudbuild.yaml` templates), set up dev/test/prod projects with the GCP admin, add monitoring and alerts, ship first small enhancement myself.
- **Days 61–90:** deliver one new internally built app or AI feature end to end, publish the golden-path starter template, define the hiring profile and onboarding guide for the next developers, present a roadmap of which vendor apps to take over next.

Red flags to mention you'd look for: no tests, secrets in Git, vendor-owned cloud accounts, manual deployments, no staging, one giant file, unpinned dependencies, LLM prompts buried in code with no evaluation.

Likely interview questions

Answer each out loud in under 2 minutes. Structure: direct answer → how you'd do it → trade-off → a real example from your career.

Technical: backend and architecture

1. **FastAPI vs Django — when each?** FastAPI: APIs, async I/O, AI services, auto OpenAPI. Django: data-heavy internal apps needing admin, ORM, auth. Both fine; consistency across the team matters more.
2. **How do you design a REST API?** Resources as nouns, correct verbs and status codes, versioning, pagination, filtering, idempotency keys for POST, consistent error format, OpenAPI contract, auth via OAuth2/OIDC tokens.
3. **SQL or NoSQL for trade data?** SQL (Postgres/Cloud SQL/AlloyDB): transactions, constraints, joins, audit. NoSQL (Firestore) for sessions, configs, chat history. BigQuery for analytics/reporting.
4. **Async in Python: when does it help?** I/O-bound work (HTTP calls, LLM calls, DB). Not CPU-bound; use processes or jobs for that. Never call blocking code inside `async def`.
5. **How do you secure an internal app on GCP?** SSO via IAP or OIDC, role-based authorization in the API, least-privilege service accounts, Secret Manager, private DB with no public IP, VPC connector, dependency scanning, audit logs.
6. **An API got slow. How do you troubleshoot?** Check metrics (latency percentiles, error rate, instance count), traces to find the slow span, logs; usual causes: N+1 queries, missing index, cold starts, connection pool exhaustion, slow downstream (LLM). Fix, then add an alert.

Technical: GCP and DevOps

1. **Why Cloud Run over GKE?** No cluster ops, scale to zero, per-request billing, revisions + traffic splitting. GKE when you need long-running stateful workloads, GPUs at scale, service mesh, or fine network control.
2. **Cold starts on Cloud Run?** Min instances, smaller images, lazy imports, startup CPU boost, keep connection pools warm.
3. **Walk me through your CI/CD pipeline.** PR → lint + unit tests → merge to main → Cloud Build: test, build image tagged with commit SHA, vulnerability scan, push to Artifact Registry, deploy to dev, integration tests, manual approval, deploy to prod with canary traffic, automatic rollback on error spike.
4. **How do you manage dev/test/prod?** Separate GCP projects, same Terraform modules with different variables, same image promoted between environments (build once, deploy many), config via env vars and Secret Manager.
5. **What is Terraform state and why remote?** State maps code to real resources; store it in a GCS bucket with locking so the team shares one truth and never edits it by hand.

6. **How do you do monitoring?** Structured JSON logs with request IDs, Cloud Monitoring dashboards, SLOs (e.g. 99.5% under 1s), alerts to the team channel, uptime checks.

Technical: AI

1. **Explain RAG end to end.** Ingest → chunk → embed → store vectors; at query time embed the question → retrieve top-k (hybrid + rerank) → build a prompt with context → model answers with citations → log and evaluate.
2. **RAG vs fine-tuning?** RAG for knowledge that changes and needs citations and access control. Fine-tuning for style, format, or narrow tasks. Most enterprise cases: RAG first.
3. **How do you stop hallucinations?** Grounding with retrieval, "say I don't know" instructions, structured output with Pydantic validation, citations checked against sources, evals, human review for high-stakes outputs.
4. **What is an AI agent and what are the risks?** An LLM looping through tool calls. Risks: wrong actions, prompt injection from documents, cost loops, data leakage. Controls: read-only tools by default, human approval for writes, max steps, per-user permissions passed to tools, full step logging.
5. **How do you test an LLM feature?** A golden set of questions with expected answers, automated scoring (exact match, similarity, LLM-as-judge), run in CI on every prompt or model change, track cost and latency too.
6. **Document access control in RAG?** Store permissions as metadata on chunks, filter at retrieval by the user's groups, never rely on the prompt to hide data.

System design (practise one on paper)

"Design an AI assistant for traders that answers questions about contracts and current positions."

Cover: React UI behind IAP → FastAPI on Cloud Run → (a) RAG over contracts in Cloud Storage, vectors in AlloyDB/pgvector with permission metadata, (b) tools calling the ETRM API for positions (read-only), (c) Gemini via the Agent Platform. Pub/Sub for document ingestion events. Logging of every prompt/tool call for audit. Evals in Cloud Build. Data residency in `me-central2`. Cost control: Flash by default, caching, limits per user.

Second one to rehearse: **"Automate extraction of trade confirmation PDFs and match them to booked trades"** (Cloud Storage upload event → Eventarc → Cloud Run extractor using Gemini structured output → compare to ETRM → mismatches to an ops review queue).

Behavioral (prepare 6 STAR stories)

Story	Use it for
Took over code someone else wrote and fixed it	Vendor transition, troubleshooting
A production incident you resolved	Ownership, calm under pressure
A system you designed end to end	Architecture, full-stack
Set up CI/CD or standards for a team	Engineering capability building
Disagreed with a vendor/stakeholder and resolved it	Collaboration, influence
Mentored a junior developer	Mentoring future hires

STAR = Situation, Task, Action (most time here, say "I"), Result (with a number).

Questions they may ask about fit

- **Why leave vendor-style work / your current job for a foundational role?** You want to build an engineering capability, not just tickets.
- **How would you handle a vendor who resists handover?** Escalate through contract terms calmly, keep the relationship, make the acceptance criteria objective.
- **You'll be the only engineer at first. How do you prioritize?** By business criticality and risk: stabilize critical apps first, quick wins second, new builds third; agree priorities with the product owner monthly.
- **No trading experience?** Be honest, then show you've learned the trade lifecycle and Front/Middle/Back office this weekend.

Questions to ask them

Pick 3–4; they show you think like the future tech lead.

- How many vendor-built applications are in scope, and which is the most business-critical?
- Do you own the repositories and GCP projects today, or does the vendor?
- What is the current deployment process, and how often do you release?
- Which AI use cases are live or planned, and how are they evaluated?

- How does the GCP administrator's role split from mine on infrastructure and access?
- What does success look like for this role after 6 and 12 months?
- What is the plan and timeline for hiring more developers?
- Which ETRM or core systems will the new apps integrate with?

Final-hour checklist

- [ ] Demo URL opens and `/v1/ask` + `/v1/agent` respond (cold start warmed up)
- [ ] Repo link ready, README with an architecture diagram
- [ ] Can draw: trade lifecycle, CI/CD pipeline, RAG flow, your system design answer
- [ ] 2-minute intro rehearsed: 10+ years → ownership → GCP shipping → practical AI → why this role
- [ ] 6 STAR stories with numbers
- [ ] 30-60-90 plan in 3 sentences
- [ ] Know the Vertex AI → Gemini Enterprise Agent Platform rename
- [ ] Questions to ask written down

Sources

- Google Cloud Next 2026: Vertex AI rebranded as Gemini Enterprise Agent Platform (HPCwire)
- Info-Tech: Google Cloud Next 2026 — It's all about the agents
- Google Cloud: Agent Development Kit overview
- Google Cloud: Gemini model versions and lifecycle
- Firebase AI Logic: current Gemini models note (Gemini 3.5 Flash)
- Deploying ADK agents to Cloud Run (Google Cloud community)